AutoScientist Challenge Report:
Grounding Chart Reasoning in Point and Polygon Primitives

Arnav Grover

Purdue University • August 2, 2026

Figure 1: Series A and B cross three times. (a) The phrase "after the crossing" matches all three, so a later step can resolve it differently. (b) A point selects one crossing and a polygon encloses a region; both are coordinates.

1. Introduction

Chart question answering requires two capabilities. The model must read the geometry of the chart, and it must track which element each reasoning step refers to. Current models perform well on the first and poorly on the second.

Figure 1 shows the failure. The phrase "after the crossing" matches three crossings, and the text does not record which one the model selected. A later step that refers back to it resolves the phrase again, and can resolve to a different crossing. The output stays fluent, so the error is invisible in the text. Lu et al. [1] call this the *reference gap*, distinct from the perception gap that higher input resolution addresses.

A second gap follows. Charts encode quantities as geometry, so answering a question about a magnitude means measuring something: the height of a bar, the angle of a wedge, the width of an interval. A model reasoning in text asserts the number without performing the measurement, and a reader cannot tell a measured value from a recalled one. We call this the *metrification gap*, and the two gaps motivate the two primitives in turn.

2. Visual primitives

Points close the reference gap. If a step names a location by coordinates rather than by description, the reference cannot move: after a step emits `<point x="642" y="392">`, every later step using that anchor addresses the same pixels. The anchor is also checkable, since a reader can draw it on the chart and see whether it lands on the element the step names.

Polygons close the metrification gap. A point has no extent, so it cannot support a claim about size. A region can. Once a region is written as an ordered set of vertices it becomes a geometric object and its properties follow by arithmetic: the height of a bar is the difference between its top and bottom edges, the share of a pie is the angle its wedge subtends at the centre, and area and overlap come the same way. The measurement stops being something the model asserts and becomes something the coordinates entail.

This is why the region primitive is a polygon and not a box. A pie wedge fills about half its bounding box, so a box naming one wedge also covers two others and any angle computed from it is meaningless. A stacked segment has a different series above and below it, so a box around it implies the wrong height, and the shaded interval in Figure 1 is bounded by two curves that no rectangle follows. A polygon is only reliable if its vertices are derived rather than estimated, so we fix one rule per element class and require two reference points per axis before any primitive is placed (Appendix C).

3. Methodology

We build on ChartQA [2]: 32,719 questions over 21,945 charts, of which 9,608 are human-authored and 23,111 machine-generated, split into 28,299 training, 1,920 validation and 2,500 test questions.

We adapt 5,000 examples from the training split with the Adaptive Data product on the Adaption platform, which compares an uploaded dataset against the others held on the platform and augments it. The input is the source triple of prompt, completion and image context. The product rewrites the prompt to remove ambiguity and the completion so that it states its steps rather than only its answer, both in house at Adaption Labs, and generates a reasoning trace following the system prompt of Appendix C. Filtering removes examples the platform could not ground, leaving 4,968 rows. Conformance to the specification rises from 7.0 to 8.9, a relative gain of 27.1 percent.

Training runs through AutoScientist, which selects the fine-tuning recipe automatically and chose LoRA at rank 16 over Gemma 4 31B-IT [3] for two epochs across 86 steps (Appendix B). We serve the adapter on one A100 80GB through Modal, evaluate on 500 questions from the test split, and grade each completion with Claude Sonnet 5.

4. Results

We compare the adapted model against two baselines: the same base model answering directly, and the base model reasoning step by step without primitives. The second baseline is needed to show that the gain comes from the grounded traces and not from step-by-step reasoning.

Model	Numeric	Other	Overall
Gemma 4 31B-IT, direct	73.5	88.9	79.6
Gemma 4 31B-IT, reasoning	80.1	88.4	83.4
Adapted (ours)	85.8	94.9	89.4

Table 1: Accuracy (%) on 500 ChartQA test questions. Numeric covers the 302 whose answer is a number, Other the other 198.

Adaptation adds 9.8 points overall, but the gain is uneven. Numeric questions improve by 12.3 points and the rest by 6.0. Step-by-step reasoning splits the same way and more sharply: it adds 6.6 points on numeric questions and loses 0.5 elsewhere. Numeric questions are the ones that require measuring the chart, so this is where the grounded traces help.

The primitives here are a training-time signal. They shape what the model learns to attend to, but the deployed model answers in prose. The next step is to elicit them at test time so that each answer carries its own coordinates and can be checked against the chart, then measure that setting against the numbers above.

Accuracy alone understates how much changes. Compared question by question with the reasoning baseline, the adapted model fixes 47 answers and breaks 20; compared with direct answering it fixes 61 and breaks 15. The headline gain is a net figure, not a uniform improvement.

We do not report the AutoScientist internal evaluation. Its judge, Gemini 3.1 Pro, scores the completion against the system prompt and returns a win rate of 1 against 99 over 200 samples. The system prompt asks for primitives, but the completion is the answer field, not the reasoning chain, and never contained primitives by design. The judge is penalising the wrong field, so we ran our own evaluation on ChartQA instead.

A. Worked example

One row of the adapted corpus, reproduced as generated, with its primitives drawn on the source chart in Figure 2.

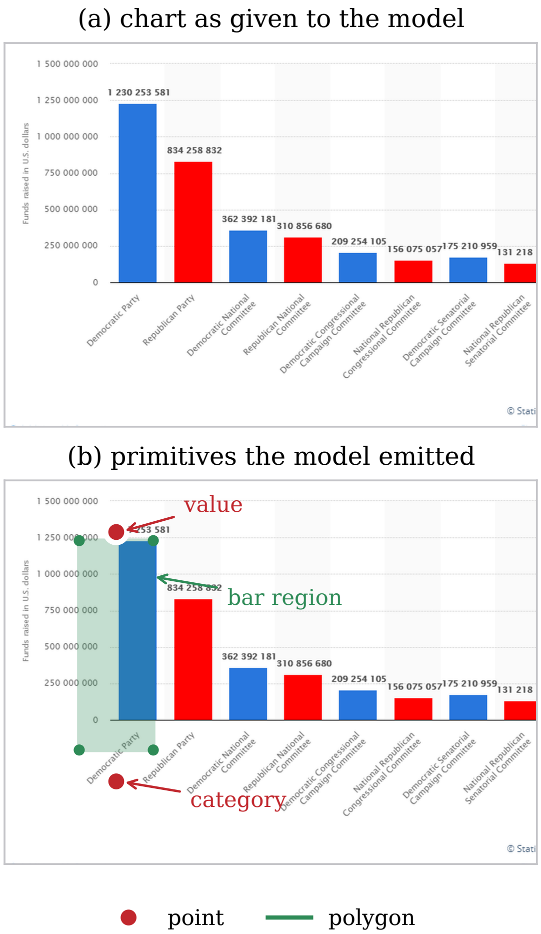

Figure 2: The chart as given to the model, and the primitives it emitted. Both points land on their targets. The polygon matches the bar on its top and right edges, which carry the value, and overshoots on the left edge and below the baseline.

Prompt

```
How much money did the Democratic Party
spend during the 2016 election cycle?
```

Completion

```
Based on the provided bar chart, the total
funds raised by the Democratic Party during
the 2016 election cycle is 1,230,253,581.
This figure is represented by the tallest
blue bar on the far left of the chart,
labeled "Democratic Party," with the
specific numerical value displayed directly
above it.
```

Reasoning chain

```
I need to locate the bar corresponding to
the "Democratic Party" on the x-axis and
extract its associated value. Scanning the
labels at the bottom, the first category on
the left is "Democratic Party"
<point x="206" y="736">Democratic Party
label</point>. Above this label stands the
tallest blue bar in the chart
<polygon points="146,176 266,176 266,663
146,663">Democratic Party bar</polygon>.
The specific numerical value is printed
directly on top of this bar, reading
"1 230 253 581" <point x="206" y="156">
Democratic Party value</point>.
Interpreting the spacing as thousand
separators, this confirms the total funds
raised. The total funds raised by the
Democratic Party during the 2016 election
cycle is 1,230,253,581.
```

B. Training configuration

Base model `google/gemma-4-31B-it-VLM`.

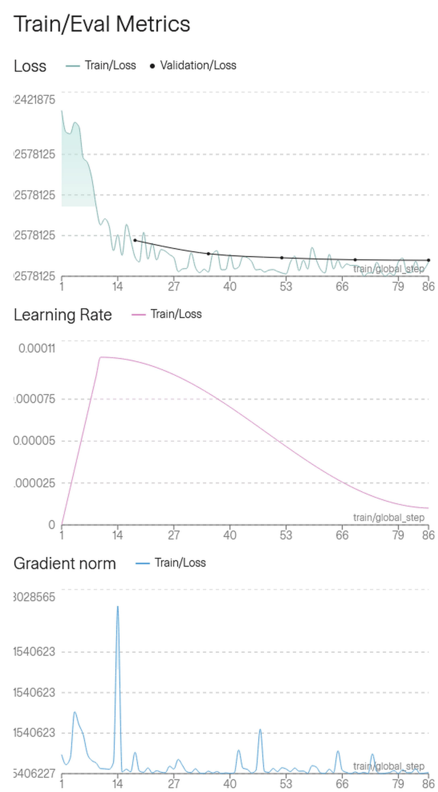

Figure 3: Training run selected by AutoScientist: loss, learning rate and gradient norm over 86 steps. Training loss falls sharply over the first 14 steps and validation loss tracks it without separating.

Evaluation covers 500 of the 2,500 ChartQA test questions, graded by Claude Sonnet 5 rather than by exact match, so the numbers carry the judge's error as well as the model's. The adapted model ran on Modal; the two baselines ran through the Google AI Studio free API.

Adapted dataset:
https://huggingface.co/datasets/Arnav-Gr0ver/Adapted_ChartQA_Visual_Primitives

Adapted weights:
https://huggingface.co/Arnav-Gr0ver/Gemma4-31B-IT-ChartQA-Adapted

Inference and evaluation scripts:
https://github.com/Arnav-Gr0ver/Visual-Primitives-ChartQA

Training method	SFT, LoRA
LoRA rank r	16
LoRA α	32
LoRA dropout	0
Trainable modules	q,k,v,o_proj
Learning rate	1×10^{-4}
Scheduler	cosine
Scheduler cycles	0.5
Minimum LR ratio	0.1
Warmup ratio	0.1
Weight decay	0.01
Max gradient norm	1
Epochs	2
Optimiser steps	86
Batch size	max
Evaluations	5
Train on inputs	false
Adapted training examples	4,968
Test questions evaluated	500
Inference hardware	A100 80GB
Mean latency per question	10.3 s

Table 2: Hyperparameters.

C. Trace generation prompt

Supplied verbatim to the generator on the Adaption platform.

```
## Role
You generate reasoning traces for chart
question answering. You receive a chart
image, a question, and the correct answer.
Treat the answer as ground truth, never
contradict it. Output a step by step trace
that arrives at that answer, with every
visual claim grounded in a coordinate
primitive.

## Primitives
Coordinates use a 0 to 1000 scale, origin at
top left, spanning the full image, including
any title, legend, axis labels, and margins,
not just the plotted data region.

<point x="" y="">: a single location, a data
value, a tick, a label anchor.

<polygon points="x1,y1 x2,y2 ...">: a
bounded region with 3 or more vertices.
Order vertices by walking around the shape's
centroid, either all clockwise or all
counterclockwise, so edges never cross.
Derive vertices the same way every time,
never freehand:
- Bar or box: the 4 rendered corners (value
  height as the top edge, baseline as the
  bottom edge, the bar's actual left and
  right edges).
- Pie wedge: apex at the pie's center, then
  vertices sampled every 15 to 30 degrees
  along the arc from the start angle to the
  end angle.
- Cluster of points: the convex hull of the
  actual member coordinates, nothing
  smoothed or added.
- Stretch of a line or area under a curve:
  vertices sampled along the curve at its
  existing data points, closed along the
  baseline if shading area, or offset by a
  small fixed band if just highlighting a
  stretch of line.
- Crossing or intersection between two
  lines: compute the intersection
  algebraically from the two already-placed
  point primitives bracketing it on each
  side (linear interpolation between
  consecutive points on each line).
  Crucially, the bracketing points must be
  actual, visible data vertices plotted on
  the lines, not assumed values for an empty
  x-axis tick. Center a small polygon on
  that computed point, giving it a
  half-width of roughly 10% of the gap
  between the bracketing x-anchors, so it
  stays comparable in size to a single data
  point rather than a sprawling region.
  Confirm the computed x actually falls
  between the two bracketing x-anchors
  before using it. Never eyeball this
  position: with both bracketing points
  already placed, the intersection is
  arithmetic.

Only place a primitive where something is
actually visible or algebraically derivable
from what's visible.
```

```
## Method
Internally, work backward: find the minimal
chart elements needed to prove the answer,
and locate them.

Before placing any primitive, locate at
least two reference points per axis directly
from the image (the axis's min and max
gridlines, or its first and last tick marks)
and record their full-image coordinates.
Interpolate every other primitive, point or
polygon, from those anchors rather than
estimating freehand. If a computed point
doesn't land on its anchor as expected (for
instance the final data point should reach
the stated axis end but doesn't), recompute
rather than emit the mismatch.

Verify Data Alignment: Before claiming a
point exists for a specific x-axis category
(e.g., a specific year), visually trace
straight down to the axis tick. If a numeric
label is offset, or if a line segment skips
an x-axis tick entirely, map the point to
its true x-axis coordinate, not an
assumption of sequential order.

Then write forward: present the trace as if
discovering the answer for the first time,
extracting values as they naturally come up,
each backed by a primitive. Never write
phrases like "since the answer is."

## Format
Plain prose with inline primitives. No extra
headers, no commentary on your own process.
```

```
One primitive per visual claim. Length
matches the question, usually a few
sentences. End with a sentence stating the
answer verbatim.

## Rules
Never invent a coordinate. Do not
hallucinate data points on x-ticks where the
line simply passes through without a plotted
vertex. If the chart cannot support the
given answer, say so instead of forcing a
trace.
```

```
## Examples

Question: Which quarter had the highest
revenue?
Answer: Q3
Trace: The chart shows four quarterly bars.
Q1, Q2, and Q4 top out around <point x="150"
y="500">Q1</point>, <point x="330"
y="430">Q2</point>, and <point x="700"
y="600">Q4</point>. Q3’s bar <polygon
points="460,180 580,180 580,850 460,850">Q3
bar</polygon> rises well above the rest. Q3
had the highest revenue.

Question: How many intercepting points are
there?
Answer: 1
Trace: The axes run from <point x="111"
y="775">2010, 0%</point> to <point x="921"
y="239">2015, 100%</point>, so every value
interpolates between those anchors. Good
sits above Bad through 2013, for instance at
<point x="595" y="459">2013 Good,
59%</point> versus <point x="595"
y="555">2013 Bad, 41%</point>. By 2014 the
order flips: <point x="760" y="604">2014
Good, 32%</point> versus <point x="760"
y="416">2014 Bad, 67%</point>. Interpolating
between the 2013 and 2014 points on each
line places a single crossing at <polygon
points="635,491 667,491 667,524
635,524">crossing region</polygon>, and the
lines don’t swap order again through 2015.
There is 1 intercepting point.
```

References

[1] R. Lu, Y. Ma, et al. Thinking with Visual Primitives. Preprint, DeepSeek-AI, Peking Univ., Tsinghua Univ., 2026. Withdrawn.

[2] A. Masry, D. X. Long, J. Q. Tan, S. Joty, E. Hoque. ChartQA: A Benchmark for QA about Charts with Visual and Logical Reasoning. *Findings of ACL*, 2022.

[3] Gemma Team, Google DeepMind. Gemma 4 Technical Report. arXiv:2607.02770, 2026.